

Cahiers des Charges EpiTrello

▼ 1. Objectifs du projet

Le projet **EpiTrello** a pour objectif de concevoir une **application web de gestion de projets** basée sur la **méthode Kanban**, permettant aux utilisateurs d'organiser et de suivre leurs tâches de manière claire, visuelle et collaborative.

- Mettre en place une interface **Kanban** avec des colonnes représentant les étapes du flux de travail.
- Permettre la **création**, **modification** et **partage** de tableaux de projets.
- Favoriser la **collaboration** en **temps réel** entre plusieurs membres.
- Offrir un **outil intuitif**, **moderne** et **responsive**.
- Assurer la **sécurité** et la **confidentialité** des données.

▼ 2. Cible du public

EpiTrello ne vise **aucun public spécifique**. Il s'adresse à un large éventail d'utilisateurs:

- Des **particuliers** (organisation personnelle)
- Des **étudiants** (travaux de groupe)
- Des **équipes** professionnelles
- Des **associations** ou **freelances**

▼ 3. Contenu et fonctionnalités

3.1. Page d'accueil

- Présentation du concept et des fonctionnalités principales.
- Boutons : **S'inscrire** / **Se connecter**.

3.2. Page d'inscription / connexion

- Formulaire d'inscription (email, mot de passe).

- Connexion via **email** ou **Google, Apple, Slack et Microsoft**

3.3. Tableau de bord

- Liste des projets de l'utilisateur.
- Bouton "Créer un tableau".

3.4. Tableau Kanban

- Colonnes personnalisables : **À faire, En cours, Terminé.**
- Glisser-déposer des cartes.
- Chaque **carte** contient : **titre, description, membres, labels, échéance, pièces jointes, checklists, commentaires.**

3.5. Profil utilisateur

- Paramètres personnels
- **Thème** clair/sombre
- **Gestion du mot de passe.**

▼ 4. Fonctionnalités techniques

- Création et gestion de tableaux, **listes, cartes**
- Interface **drag & drop** pour déplacer les cartes entre colonnes
- Collaboration en temps réel (membres, commentaires, notifications)
- Système d'étiquettes colorées, dates limites et checklists (**tags**)
- Mode **sombre** et mode **clair**
- Recherche globale et filtres avancés
- Authentification sécurisée via **JWT** (JSON Web Tokens)

▼ 5. La gestion des langues et autres notions régionales

- Langue principale : **Anglais**
- Langue secondaire : **Français (internationalisation i18n prévue)**
- Formats de date : **AAAA-MM-JJ (ISO 8601)**

- Fuseau horaire : détection automatique selon le navigateur de l'utilisateur

▼ 6. Plateforme de développement

6.1. Architecture Globale

Composant	Technologie
Framework Frontend	Next.js 16 (React)
Langage	TypeScript
Styling	Tailwind CSS
Framework Backend	NestJS (Node.js)
API	GraphQL (Apollo Server)
Base de données	PostgreSQL
ORM	Prisma
Temps réel	Socket.IO
Authentification	JWT (Passport.js)
Hébergement	App Engine (Frontend) + Cloud Run (Backend)

6.2. Choix technologiques justifiés

6.2.1. Front-end

Next.js (React)

- Structure optimale pour le SEO et le déploiement
- Parfait pour combiner pages publiques (landing) et SPA interactive (tableaux)
- App Router moderne pour une architecture performante

TypeScript

- Réduit significativement les bugs en production
- Facilite la collaboration et le refactoring
- Indispensable pour gérer des objets complexes (**Card**, **User**, **Board**)

Tailwind CSS

- Développement **UI** rapide avec classes utilitaires
- Responsive design intégré par défaut
- Cohérence visuelle sans fichiers CSS volumineux

6.2.2. Back-end

Node.js

- Même langage que le frontend (**JavaScript/TypeScript**)
- Partage de logique et gain de productivité
- Écosystème riche pour WebSockets et APIs

NestJS

- Architecture **maintenable** pour des équipes
- Facilite les **tests** et **l'architecture modulaire**
- Modules indépendants (**auth**, **boards**, **realtime**)

GraphQL (Apollo Server)

- Requêtes **flexibles** - le client demande exactement les données nécessaires
- Réduit le nombre de requêtes et la bande passante
- **Typage** fort et documentation auto-générée
- Parfait pour gérer des relations complexes (tableaux, listes, cartes)

6.2.3. Base de données & ORM

PostgreSQL

- Base de données **relationnelle** robuste et éprouvée
- Gère parfaitement les relations complexes (utilisateurs ↔ tableaux ↔ cartes)
- Transactions nécessaires pour déplacements atomiques de cartes

Prisma

- ORM moderne pour **TypeScript** avec **typage fort**

- Génère automatiquement les **types TypeScript**
- Simplifie les **migrations** et améliore la productivité

6.2.4. Hébergement sur Google Cloud Platform

Architecture GCP

- **App Engine (Frontend)** : Plateforme managée pour Next.js avec scaling automatique
- **Cloud Run (Backend)** : API NestJS containerisée avec auto-scaling selon la charge
- **Cloud SQL (PostgreSQL)** : Base de données managée avec haute disponibilité
- **Cloud Storage** : Stockage des fichiers uploadés avec URLs signées
- **Cloud Build** : CI/CD intégré pour déploiement automatique
- **Cloud CDN** : Distribution du contenu statique à l'échelle mondiale

Avantages de GCP

- Écosystème Google intégré (Auth, Analytics, etc.)
- App Engine optimisé pour Next.js (support natif Node.js)
- Cloud Run performant pour APIs containerisées
- Tarification à l'usage (pay-as-you-go)
- Niveau gratuit généreux (App Engine, Cloud Run, Storage, SQL)
- Infrastructure mondiale avec faible latence
- Monitoring et logging intégrés (Cloud Logging, Cloud Monitoring)

6.2.5. Temps réel & fichiers

Socket.IO

- **Rooms** par tableau pour isolation des événements
- Faible latence pour synchronisation instantanée
- Facile à intégrer pour synchroniser actions et présence

Stockage fichiers (Google Cloud Storage)

- Évite les blobs lourds dans la base de données
- URLs signées pour accès sécurisé et temporaire
- Intégration native avec Cloud Run
- Solution scalable, performante et économique

▼ 7. Sécurité

La sécurité est une priorité absolue pour EpiTrello. L'application respecte les normes et réglementations en vigueur, notamment le **Règlement Général sur la Protection des Données (RGPD)**.

7.1. Conformité RGPD

- **Consentement explicite**: Les utilisateurs doivent accepter explicitement la collecte et le traitement de leurs données personnelles lors de l'inscription
- **Droit d'accès et de rectification** : Les utilisateurs peuvent consulter et modifier leurs données à tout moment depuis leur profil
- **Droit à l'oubli** : Possibilité de supprimer définitivement son compte et toutes les données associées
- **Portabilité des données** : Export des données personnelles au format JSON sur demande
- **Transparence**: Politique de confidentialité claire et accessible décrivant les traitements de données

7.2. Authentification et autorisation

- **Tokens JWT** : Authentification sans état avec tokens sécurisés, expiration configurable (7 ou 30 jours)
- **Hashage des mots de passe** : Utilisation de bcrypt avec salt pour un stockage sécurisé
- **OAuth 2.0** : Intégration avec Google, Apple, Slack et Microsoft pour une authentification externe sécurisée
- **Réinitialisation sécurisée** : Tokens temporaires à usage unique envoyés par email (expiration 1 heure)

- **Gestion des rôles** : Système de permissions granulaires (ADMIN, MEMBER, VIEWER) pour les workspaces et tableaux

7.3. Protection des données

- **Chiffrement en transit** : HTTPS/TLS 1.3 obligatoire pour toutes les communications
- **Chiffrement au repos** : Base de données PostgreSQL avec chiffrement AES-256
- **Validation des entrées** : Sanitisation systématique côté serveur pour prévenir les injections SQL et XSS
- **Protection CSRF** : Tokens anti-CSRF pour toutes les opérations sensibles
- **Rate limiting** : Limitation du nombre de requêtes pour prévenir les attaques par force brute et DDoS

▼ 8. Accessibilité

EpiTrello s'engage à offrir une expérience accessible à tous les utilisateurs, conformément aux **standards WCAG 2.1 niveau AA** (Web Content Accessibility Guidelines).

8.1. Lecteurs d'écran

- **Structure sémantique** : Utilisation appropriée des balises **HTML5** (header, nav, main, section, article)
- **ARIA labels** : Attributs descriptifs pour tous les éléments interactifs et dynamiques
- **Textes alternatifs** : Description textuelle pour toutes les images et icônes
- **Live regions** : Annonces des changements dynamiques (notifications, mises à jour de cartes)
- **Hiérarchie des titres** : Structure logique H1-H6 pour faciliter la navigation

8.2. Contraste et lisibilité

- **Ratio de contraste** : Minimum 4.5:1 pour le texte normal, 3:1 pour le texte large (WCAG AA)

- **Taille de police** : Minimum 16px pour le corps de texte, possibilité d'agrandir jusqu'à 200%
- **Thèmes adaptatifs** : Mode clair et sombre avec contrastes optimisés pour chaque
- **Espacement** : Marges et paddings suffisants pour éviter la confusion visuelle

8.3. Interactions et formulaires

- **Labels explicites** : Tous les champs de formulaire ont des labels visibles et associés
- **Messages d'erreur** : Instructions claires et suggestions de correction
- **Zones cliquables** : Minimum 44×44 pixels pour tous les boutons et liens (règle du pouce)
- **Drag & drop alternatif** : Menus contextuels et boutons pour déplacer les cartes sans glisser-déposer
- **Timeouts ajustables** : Aucune limite de temps stricte pour compléter les actions

8.4. Internationalisation accessible

- **Attribut lang** : Indication de la langue pour chaque page et changements de langue
- **Direction du texte** : Support RTL (right-to-left) pour langues arabes et hébraïques prévu
- **Formats localisés** : Dates, heures et nombres adaptés à la locale de l'utilisateur

8.5. Tests et conformité

- **Tests automatisés** : Intégration d'outils comme axe-core et Lighthouse dans le pipeline CI/CD
- **Tests manuels** : Validation avec lecteurs d'écran (NVDA, JAWS, VoiceOver)
- **Tests utilisateurs** : Sessions avec utilisateurs en situation de handicap

- **Documentation** : Guide d'accessibilité mis à jour avec chaque nouvelle fonctionnalité

▼ 9. Maintenance et mises à jour

Le site nécessitera des mises à jour régulières pour assurer la sécurité, la compatibilité et l'ajout de nouvelles fonctionnalités.

- **Architecture modulaire**: Grâce à **React** et **Next.js**, la maintenance est facilitée par une structure en composants réutilisables
- **Version control**: Gestion via **GitHub** avec branches protégées et revue de code obligatoire
- **CI/CD** : Déploiement continu sur **Vercel** garantissant des mises en ligne rapides et sans interruption
- **Tests automatisés**: Suite de tests unitaires et d'intégration pour prévenir les régressions
- **Monitoring**: Surveillance en temps réel des performances et des erreurs en production
- **Documentation technique**: Mise à jour continue de la documentation API et architecture

API Endpoints

Trello Features

▼ 10. Endpoints API GraphQL

L'**API GraphQL** expose les opérations suivantes:

10.1. Authentification

- **register** : Création de compte utilisateur
- **login** : Connexion avec email/mot de passe
- **forgotPassword** : Demande de réinitialisation de mot de passe
- **resetPassword** : Réinitialisation avec token

10.2. Utilisateurs

- **me** : Profil de l'utilisateur connecté
- **user(id)** : Détails d'un utilisateur spécifique

- **updateUser** : Modification du profil
- **deleteUser** : Suppression de compte

10.3. Tableaux (À venir)

- **boards** : Liste des tableaux accessibles
- **board(id)** : Détails d'un tableau
- **createBoard** : Création de tableau
- **updateBoard** : Modification de tableau
- **deleteBoard** : Suppression de tableau

10.4. Cartes (À venir)

- **card(id)** : Détails d'une carte
- **createCard** : Création de carte
- **updateCard** : Modification de carte
- **moveCard** : Déplacement entre listes
- **deleteCard** : Suppression de carte